

Received March 18, 2021, accepted March 28, 2021, date of publication April 2, 2021, date of current version April 22, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3070690

An End-to-End Multiplex Graph Neural Network for Graph Representation Learning

YANYAN LIANG¹, YANFENG ZHANG¹, (Member, IEEE), DECHAO GAO, AND QIAN XU

Department of Computer Science and Engineering, Northeastern University, Shenyang 110819, China

Corresponding author: Yanfeng Zhang (zhangyf@mail.neu.edu.cn)

This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFB1003404, in part by the National Natural Science Foundation of China under Grant 62072082, Grant 61672141, and Grant U1811261; in part by the Fundamental Research Funds for the Central Universities under Grant N181605017 and Grant N181604016, and in part by the Key Research and Development Program of Liaoning Province under Grant 2020JH 2/10100037.

ABSTRACT Research on graph classification tasks based on graph neural networks has attracted wide attention. The graphs to be classified may have various graph sizes (i.e., different numbers of nodes and edges) and have various graph properties (e.g., average node degree, diameter, and clustering coefficient). The diverse property of graphs has imposed significant challenges on existing graph learning techniques since diverse graphs have different best-fit hyperparameters. Consequently, it is unreasonable to learn graph representation from a set of diverse graphs by a unified graph neural network. Inspired by this, we design an end-to-end Multiplex Graph Neural Network (MxGNN) that learns graph representations with multiple GNNs, and combines them with a learnable method. The main challenge lies with the combination of multiple representation results. Our new findings show that the a priori graph properties do have an effect on the quality of representation learning, which can be used to guide the learning. Our experiments on graph classification with multiple data sets show that the performance of MxGNN is better than the existing graph representation learning methods.

INDEX TERMS Graph neural networks, graph classification, graph representation learning.

I. INTRODUCTION

Graphs are known to have complicated structures and have a myriad of real-world applications. Recently, great efforts have been put into utilizing deep learning methods for graph data analysis. Many newly proposed graph learning approaches are inspired by Convolutional Neural Networks (CNNs) [1], which have been greatly successful in learning two-dimensional image data (grid structure). The convolution layers in CNNs have been redefined to process graph data. A multitude of different Graph Convolutional Networks (GCNs) [2] have been proposed, which can learn node level representations by aggregating feature information from neighbors (spatial-based approaches) [3] or by introducing filters from the perspective of graph signal processing (spectral-based approaches) [4].

However, the diverse properties of graphs have imposed significant challenges on existing graph representation learning techniques. The graphs to be learned have various graph

sizes (i.e., different numbers of nodes and edges) and have various graph properties (e.g., average node degree, diameter, etc.). Using a unified graph neural network with consistent hyperparameters can bring troubles in graph representation learning. For example, when performing node-representation learning tasks (by graph convolution operation), it is enough to use small output embedding size for simple and small graphs, as shown in Figure 1a, since large embedding size could result in an overfitting problem. By contrast, it is necessary to set large output embedding sizes for complex and large graphs to learn complex graph structure properties, as shown in Figure 1b.

To verify our analysis, we divide the PROTEINS graphs dataset [5] into three subsets of graphs evenly according to their sizes (number of nodes) and run a state-of-the-art graph hierarchical representation learning model DiffPool [6] on the three subsets of graphs respectively to perform the graph classification task. The average accuracy results with different embedding size parameters ($dim = 10, 60, 120$) on the three subsets are listed in Table 1. We can see that small graphs prefer small embedding sizes while large graphs prefer large

The associate editor coordinating the review of this manuscript and approving it for publication was Essam A. Rashed¹.

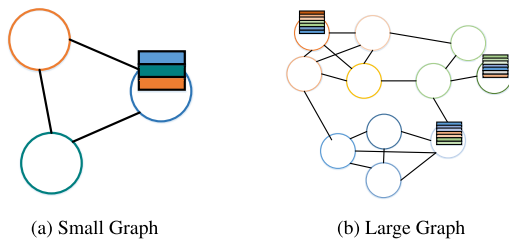


FIGURE 1. Multiple graph convolution networks with different hidden/output node embedding sizes are needed for graphs with various sizes. A small graph requires a small node embedding size, while a large graph requires a large node embedding size.

TABLE 1. Effect of embedding size (dim) on various-size graphs (DiffPool [6] on PROTEINS dataset).

graph size	dim=10	dim=60	dim=120
small	74.70	73.91	73.55
medium	71.56	73.72	71.47
large	81.32	81.57	82.72

TABLE 2. Effect of embedding size (dim) on various-size graphs (DiffPool [6] on NCI109 dataset).

graph size	dim=20	dim=50	dim=80
small	83.44	82.50	79.79
medium	74.67	76.85	75.44
large	82.57	83.47	85.45

embedding sizes as expected, and we obtain better results on large graphs. Furthermore, we also observe similar trends on the NCI109 dataset as shown in Table 2.

We investigated the latest GNN/GCN-related literature, such as [5] says that existing GCN methods generally require a wealth of domain knowledge to obtain the data graph, which cannot guarantee that the graph is suitable. References [6] says that most of GNNs have a fixed architecture and cannot change their components to adapt to different tasks, etc. But none of the existing models have solved the problem we proposed. The diversity of graphs dataset and its effect on the preference on various GNN hyperparameter settings motivate us to use multiplex GNN structure to learn graph features in a diverse way. As known, a common solution for augmenting the traditional CNN convolution layers is to use multiple convolution kernels in order to learn multiple local features. The success of CNNs on image data also inspires us to concurrently use multiple graph convolution networks to learn graph representations. Besides, the effect of graph size on the performance also motivates us to utilize *a priori* properties of graphs to guide the learning process.

In this paper, we propose MxGNN¹ in graph representation learning for graph classification tasks. MxGNN comprises multiple graph convolution networks (with different hyperparameters) to learn node-level representations and uses graph pooling networks to coarsen the graph. The node-level representations resulted from multiple convolution networks are merged in a learnable way. The *a priori* properties of graphs are considered during the merge process, where the to-be-merged vectors/matrices from different GNNs are endowed

with different weights. The merged node representations and the coarsened graph are then used to generate a new coarsened graph, which is used in the next layer. This multiplex structure can adapt to graphs with diverse properties and can extract useful information from different perspectives. We conduct extensive experiments on numerous graph classification benchmarks. Our results demonstrate that MxGNN can achieve superior performance over the state-of-the-art methods GraphSAGE [3], GCN [7], SortPool [8], DiffPool [6], SAGPool [9], gPool [10], and GIN [11] on five of six graph classification datasets. We also launch experiments to demonstrate the effect of multiplex convolution networks by comparing multiplex convolution neural networks with single convolution neural networks. We also show the attention weight distribution of multiple convolutions after the training is converged. The results demonstrate that MxGNN indeed learns different attention weights of multiple networks for different size graphs. This verifies our idea that multiple networks with different hyperparameters are appropriate for different size graphs.

The rest of this paper is organized as follows. In Section II, we review the literature of GNNs. We describe MxGNN in detail in Section III. We show our experimental results in Section IV. We conclude our paper in Section V.

II. RELATED WORK

In this section, we first introduce the graph classification task and then review the recent literature on graph neural networks, graph convolution methods, and graph pooling methods.

A. GRAPH CLASSIFICATION

Graph classification is to identify the class labels of graphs in a dataset, which is an important problem with practical applications in bioinformatics, chemoinformatics, social network analysis, cyber-security, etc. For example, in chemoinformatics, molecules can be represented as graphs where nodes correspond to atoms, and edges correspond to the presence of chemical bonds between pairs of atoms [12]. The molecules with different atom-level structures (different graphs) have different properties (graph labels), such as anti-cancer activity, solubility, and toxicity. To handle graph classification, as a popular technique, Shervashidze *et al.* [13] proposed to count the occurrences of various graphlets (i.e., subgraphs) on a graph. In recent years, the focus has shifted towards learning data-driven graph features [12], [14] where task-relevant features are learned automatically from the graphs in a given dataset. SCNN [15] first design a comprehensive weighting method to measure the significance of vertices in the graph based on multiple indicators to choose the central node sequence, then construct the same size graphs for the central vertices, and extract characteristics of the graph from both local and global aspects. We can then use the calculated graph features for classification. On the other hand, graph neural networks and graph-level representation learning methods [6], [9], [10], [16] on graph data structures

¹Our code is available at <https://github.com/JucatL/MxGNN/>.

that automatically perform an end-to-end learning process on the given dataset have attracted more and more attention. In this paper, we focus on improving the existing graph-level representation learning methods and provide more accurate graph classification results.

B. GRAPH CONVOLUTION/POOLING

Graph Neural Networks(GNN) have recently drawn considerable attention due to their superiority in a wide variety of graph-related tasks, including node classification [7], link prediction [17], and graph classification [18]. References [19] review the thirty-year history of ML by elaborating on supervised learning, unsupervised learning, reinforcement learning, and deep learning. Many of these GNN models are inspired by traditional learning techniques. Inspired by the deep learning, [20] introduces a deep graph neural network-based social recommendation framework(GNN-SoR), [21] introduces a deep learning-embedded social Internet of Things for an ambiguity-aware social recommendation. Inspired by autoencoder, [22] introduces a graph embedding-based generative adversarial network. Inspired by the huge success of convolutional networks in the computer vision domain, a large number of graph convolutional networks have emerged. The key to graph convolution is to learn a function to generate a node's representation by aggregating its features and neighbors' features. References [4] first introduces convolution for graph data from the spectral domain using the graph Laplacian matrix L . Besides, there exist numerous spectral-based graph convolution methods, such as ChebNet [23], 1stChebNet [7], and AGCN [24]. In contrast, spatial-based convolution methods define graph convolution based on a node's spatial relations. To explore the depth and breadth of a node's receptive field, multiple graph convolution layers are stacked together, so that the features of two or more hops away neighbors can be learned. For example, GCN [7], GGNNs [25], MPNN [26], and GraphSage [3] all fall into the spatial-based category. GCN is applied to many models. GARG [27] implement a collaborative preference module to learn the embeddings as the user attributes over the collaborative information, where GCN is leveraged upon the content information, i.e., distance, to adaptively identify the correlations. IIoT cloud platform supports data sharing, which can reduce data storage and improve computing speed [28]. GCN also has a way to improve the calculation speed, that is, GCN has the characteristic of parameter sharing. Because of the different convolution methods of spectral-based and spatial-based, the way of parameter sharing is different. Parameter sharing can reduce the number of parameters, so it can improve the calculation speed. It is very important for convolution operation, which is also one of the advantages of GCN.

Graph pooling operation is of vital importance for graph classification tasks [8]. It coarsens a graph into sub-graphs [6], [23] or to sum/average over the node representations [12], [26], which can obtain a compact representation on graph level. DiffPool [6] was the first to propose learned graph pooling. It learns a soft cluster assignment matrix in layer l

which contains the probability values of nodes being assigned to clusters. A cluster in layer l will be reduced to a node in layer $l + 1$. A GNN with input node features and adjacency matrix is used to generate the soft assignment matrix, based on which we can learn the cluster embeddings (i.e., node features in the next layer) and the coarsened adjacency matrix denoting the connectivity strength between each pair of the clusters.

Besides DiffPool, numerous graph pooling methods have emerged recently. SortPool [8], gPool [10], and SAGPool [9] learn to select the top k important nodes from the original graph (instead of averaging the clustered nodes) to form the coarsened graph. However, the ignored nodes may contain important features and this information is lost during pooling. Analogous to the classical signal processing, EigenPooling [29] defines graph Fourier transform and spectral representation of the graph signal in the spectral domain, then it transforms a graph signal defined on a given graph to a corresponding graph signal defined on the coarsened version of this graph, which forms the hierarchical coarsening process. Relational Pooling (RP) [30] provides a framework with maximal representation power for graphs, which can make WL-GNNs more expressive than the WL test. StructPool [31] formulates the node-cluster assignment as a structured prediction problem and employs conditional random fields to capture the relationships among the assignments of different nodes. However, to the best of our knowledge, none of the existing pooling methods employs a multiplex structure to learn graph representations in a diverse way.

III. METHODOLOGY

In this section, we propose MxGNN to learn graph representations for graph-level classification tasks. Before going to the details, we first introduce some notations and the problem settings.

Notations A graph can be represented as $\mathcal{G} = \{A, F\}$, where $A \in \mathbb{R}^{n \times n}$ denotes the adjacency matrix (n is the number of nodes contained in $\mathcal{G}$), and $F \in \mathbb{R}^{N \times d}$ denotes the node feature matrix (d is the dimension of features). In the graph classification setting, given a set of graphs and each being associated with a label, we aim to train a model that takes an unseen graph as input and predicts its corresponding label. To make the prediction, it is important to extract useful information from multiple perspectives including both graph structure and node features.

A. OVERVIEW

MxGNN is a multi-layer hierarchical GNN model. At each layer, MxGNN consists of convolution operation and pooling operation. The convolution operation aims to learn node-level representations, while the pooling operation aims to learn a coarsened graph. The new coarsened graph can then be used as input to the next layer. This process can be repeated several times, generating a multi-layer GNN model to learn hierarchical graph representations. To simplify the illustration, we choose GCN [7] as the convolution layer and

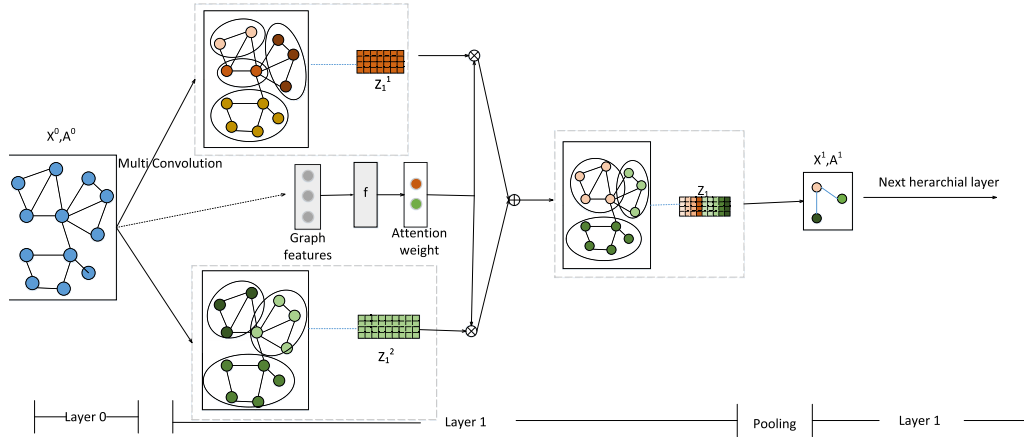


FIGURE 2. An illustrative example of MxGNN. At each hierarchical layer, we run multiple GCNs with different hyperparameters on the original graph, so we will have multiple diverse sets of node embeddings. The a priori graph properties are used to guide the merge of different sets of node embeddings. The better-fit GCN's results are given higher weight (by ' $\otimes$ '), where the weights are learned by a softmax layer. The node embeddings from different GCNs are concatenated (' $\oplus$ ') as the input of a fully connected layer, which will output the merged node embeddings. We also run differentiable pooling operations on the original graph, and cluster nodes together to obtain node-cluster assignment matrices. The merged node embeddings and the assignment matrix are used to generate a new coarsened graph with a new set of node embeddings, which is going to be processed in the next hierarchical layer.

DiffPool [6] as the pooling layer (which is a differentiable pooling method), but it can be extended to use other convolution/pooling variant as well.

Different from other hierarchical GNNs, MxGNN launches multiple GCNs to learn node-level representations. The node-level representations resulted from multiple GCNs are then merged in a learnable way by considering the a priori graph properties (e.g., number of nodes/edges, diameter, average node degree, etc.). The merged node embeddings and the coarsened graph are used to generate a new coarsened graph with a new set of node features. This multiplex structure can help extract useful information from different perspectives and can adapt to graphs with different sizes. We provide an illustrative example as shown in Figure 2.

The procedure of the GCN [7] is to “horizontally” learn node representations, as it can only “pass message” between nodes through edges. The procedure of differentiable pooling [6] is to “vertically” summarize the node features into the higher-level graph representation. The procedure of multiplexing is to “diversely” learn node representations or graph representations from different perspectives. The procedure of merging is to “synthetically” learn the diverse results and put more attention to one or more perspectives according to the a priori graph features. Since the convolution, pooling, and merging operations are all differentiable, we can define an end-to-end differentiable graph representation learning framework in a hierarchical manner.

B. MULTIPLEX CONVOLUTION

In our model, we use GCN for the convolution operation. The original GCN [7] is stacked by several convolutional layers, and a single convolutional layer can be written as

$$H^{(k+1)} = \text{ReLU}(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(k)} W^{(k)}), \quad (1)$$

$H^{(k)} \in \mathbb{R}^{n \times d}$ are the node embeddings computed after k steps. $\tilde{A} = A + I$. $\tilde{D} = \sum_j \tilde{A}_{ij}$. $W^{(k)} \in \mathbb{R}^{d \times d'}$ is a trainable

weighted matrix where d' denotes the output embedding's size. Equation (1) can be understood as a message passing process. The node embeddings $H^{(k)}$ are the “messages” transferred along edges, which are going to be used to generate new node embeddings in next round. A total number of K convolutional layers are stacked to learn node representations and the output matrix $Z = H^K$ can be viewed as the final node representations learned by the GCN model.

In MxGNN, suppose there are totally L layers. At each layer l , with the input node feature matrix $X^l \in \mathbb{R}^{n^l \times d^l}$ and the adjacency matrix $A^l \in \mathbb{R}^{n^l \times n^l}$ generated from previous layer, we learn an embedding matrix $Z^{(l)} \in \mathbb{R}^{n^l \times d^{l+1}}$. Here, we use d^{l+1} to denote the output embeddings' dimension since it will determine the input node embeddings $X^{(l+1)}$ at next layer $l+1$ that will be introduced later. For simplicity's sake, we will use $Z^{(l)} = \text{GCN}_c(A^{(l)}, X^l)$ to denote the GCN process (containing K iterations of message passing). Initially, $A^{(1)} = A \in \mathbb{R}^{n \times n}$ is the original graph's adjacency matrix, and $X^{(0)} = F \in \mathbb{R}^{n \times d}$ is the original graph's node features.

In MxGNN, we use multiple GCNs to learn multiple sets of node embeddings. These GCNs can be trained with different sets of hyperparameters θ , such as weight matrix W 's dimension d'_i . Suppose there are n_c GCNs running concurrently at each layer l , we will have n_c sets of node embeddings $\{Z_1^{(l)}, Z_2^{(l)}, \dots, Z_{n_c}^{(l)}\}$. Let θ_i be the hyperparameters set of the i th GCN. Then at layer l , we have node embeddings $Z_i^{(l)}$ resulted from the i th GCN as follows:

$$Z_i^{(l)} = \text{GCN}_c(A^{(l)}, X^l, \theta_i). \quad (2)$$

Because we learn node representations from multiple aspects through multiple sets of GCN, each set of GCN contains a small number of convolutional layers (two to three layers), and the number of layers in the entire framework is also small ($L=1$ or 2). Therefore, the number of convolutional layers of

the entire model is small, which avoids the problem of over-fitting in traditional graph convolutional networks.

A learnable method for Merging We then utilize the input graph's a priori properties to merge these diverse sets of node embeddings. In our implementation, we use the number of nodes, the number of edges, and the average node degree to construct a 3-dimensional graph properties vector. Let $\mathbf{g} \in \mathbb{R}^f$ denote the a priori graph properties vector of a specific input graph where f is the number of graph properties to be considered. According to $\mathbf{g}$, we employ a softmax normalization to obtain the attention weight of each GCN:

$$\alpha_i^g = \frac{\exp(\mathbf{g}^T \cdot \mathbf{W}_i^g)}{\sum_{j=1}^{n_c} \exp(\mathbf{g}^T \cdot \mathbf{W}_j^g)}, \quad (3)$$

where $\mathbf{W}^g \in \mathbb{R}^{f \times n_c}$ is the weight matrix of a shared linear transformation which is applied to every input graph. Then, the multiple sets of node embeddings $\{Z_1^{(l)}, Z_2^{(l)}, \dots, Z_{n_c}^{(l)}\}$ are weighted first and merged into one set of node embeddings $Z^{(l)}$ using a neural network:

$$Z^{(l)} = f_c(\alpha_1^g \otimes Z_1^{(l)} \oplus \alpha_2^g \otimes Z_2^{(l)} \oplus \dots \oplus \alpha_{n_c}^g \otimes Z_{n_c}^{(l)}), \quad (4)$$

where “ $\oplus$ ” denotes a row-wise concatenation operation, “ $\otimes$ ” denotes a scalar operation, and $f_c()$ is a single-layer MLP neural network. One hyperparameter of $f_c()$ is the output embeddings' dimension d^{l+1} , i.e., $Z^{(l)} \in \mathbb{R}^{n^l \times d^{l+1}}$, which is set by averaging the dimensions of the multiple weight matrices $\{W_1, W_2, \dots, W_{n_c}\}$. Suppose $W_i \in \mathbb{R}^{n^l \times d_i^l}$, we can set $d^{l+1} = \sum_{i=1}^{n_c} d_i^l / n_c$. The node embeddings $Z^{(l+1)}$ will then be used as input to the next layer.

C. POOLING

We follow DiffPool [6] to construct our pooling layer. We learn to assign nodes to clusters at each layer l using the node embeddings and adjacency matrix generated from the previous layer. Specifically, at each layer l , we learn cluster assignment matrices S , which is generated as follows:

$$S^{(l)} = \text{softmax}\left(\text{GCN}_p(A^{(l)}, X^{(l)})\right). \quad (5)$$

It is noticeable that GCN_p is a GCN which is different from the GCN_c used in the convolution layer, though these two GNNs consume the same input data. Each row of $S^{(l)}$ corresponds to one of the n^l nodes at layer l . Given the number of nodes at the next layer $l+1$, n^{l+1} , and each column of $S^{(l)}$ corresponds to one of the n^{l+1} clusters, i.e., $S^{(l)} \in \mathbb{R}^{n^l \times n^{l+1}}$. One important hyperparameter could be the coarsening ratio that determines the number of clusters to be assigned. Different pooling networks can have a different number of clusters.

The merged node embeddings $Z^{(l)}$ as shown in Equation (4) and the assignment matrix $S^{(l)}$ as shown in Equation (5) are used to generate embeddings for each of the n^{l+1} clusters [6]. The adjacency matrix $A^{(l)}$ and the assignment matrix $S^{(l)}$ are also used to generate a coarsened adjacency matrix denoting the edge weights between each pair of cluster:

$$X^{(l+1)} = S^{(l)T} Z^{(l)},$$

$$A^{(l+1)} = S^{(l)T} A^{(l)} S^{(l)}, \quad (6)$$

Here, since $S^{(l)} \in \mathbb{R}^{n^l \times n^{l+1}}$ and $Z^{(l)} \in \mathbb{R}^{n^l \times d^{l+1}}$, we have the cluster embeddings $X^{(l+1)} \in \mathbb{R}^{n^{l+1} \times d^{l+1}}$. Similarly, we have the coarsened adjacency matrix $A^{(l+1)} \in \mathbb{R}^{n^{l+1} \times n^{l+1}}$.

Note that, the coarsened graph is a fully connected weighted graph so that the coarsened adjacency matrix $A^{(l+1)}$ is a real matrix and each entry in $A^{(l+1)}$ denotes the edge weight between two clusters. The cluster embeddings $X^{(l+1)}$ and the coarsened adjacency matrix $A^{(l+1)}$ will then be used as input to the next layer, where one cluster at layer l corresponds to one node at layer $l+1$.

IV. EXPERIMENTS

In this section, we compare MxGNN with the state-of-the-art graph representation learning methods in the context of the graph classification task.

A. EXPERIMENTAL SETTINGS

Datasets In our experiments, we use four bioinformatics protein datasets: D&D [32], ENZYMES [5], PROTEINS [5], NCI109 [33], and two social network datasets: COLLAB [34] and RDT-M12K [34]. Each of these datasets includes hundreds to thousands of graphs. The details of these datasets are provided in Table 3. The graphs exhibit great diversity in graph sizes and complexity. Table 3 also lists the min/max number of nodes/number of edges/average node degree of each dataset of graphs. The nodes in bioinformatics graphs have categorical features. As regards social graphs, whose nodes do not have features, we use an uninformative feature vector for all nodes. This helps compare all models with the same input representations.

Baselines We consider the following state-of-the-art methods for graph classification tasks as baselines: GraphSAGE [3] is a graph convolution framework proposed for semi-supervised node classification. GraphSAGE with global mean-pooling on the learned node representations can realize graph representation learning. SortPool [8] is a global pooling method that uses sorting for pooling. It is built upon the GCN layer, where the features of nodes are sorted before feeding them into traditional 1D convolutional and dense layers. gPool [10] achieves pooling operation by adaptively selecting top-k nodes to form a smaller graph based on their scalar projection values on a trainable projection vector. SAGPool [9] is a Self-Attention Graph Pooling method for GNNs in the context of hierarchical graph pooling. The self-attention mechanism is exploited to distinguish between the nodes that should be dropped and the nodes that should be retained. GIN [11] abbr. Graph Isomorphism Network, is shown to be more powerful than traditional GNNs and is as powerful as the Weisfeiler-Lehman graph isomorphism test. DiffPool [6] is the first end-to-end trainable graph pooling method that learns hierarchical representations of graphs.

Experimental Setup To remove the unwanted bias towards the training data, we use 10-fold cross-validation for all the baselines and our approach. Since DiffPool is the

TABLE 3. Statistics of data sets.

dataset	graphs	classes	[min,max] nodes	[min,max] edges	[min,max] avg-deg
D&D	1178	2	[30, 5748]	[63, 14267]	[7.22, 17.87]
ENZYMES	600	6	[2, 125]	[1, 149]	[2.00, 10.46]
PROTEINS	1113	2	[4, 620]	[5, 1049]	[3.43, 10.14]
NCI109	4127	2	[4, 111]	[3, 119]	[2.50, 5.54]
COLLAB	5000	3	[60, 492]	[60, 40120]	[13.94, 952.02]
RDT-M12K	11929	11	[2, 3782]	[1, 5171]	[4.00, 26.37]

TABLE 4. Average accuracy on graph classification.

Baselines	D&D	ENZYMES	PROTEINS	NCI109	COLLAB	RDT-M12K
BaseLine [35]	78.07	61.72	75.16	66.95	55.65	23.58
GraphSAGE [3]	72.36	45.59	70.48	76.50	68.25	42.20
SortPool [8]	78.32	31.29	73.54	70.80	73.76	31.44
gPool [10]	75.01	48.33	71.63	74.52	71.12	OOD
SAGPool [9]	76.94	43.99	72.91	72.51	79.27	43.25
GIN [11]	75.57	48.32	71.65	75.44	79.48	47.22
DiffPool [6]	80.01	62.17	75.96	80.10	71.78	47.05
MxGNN (Ours)	80.47	68.22	76.30	81.89	74.43	47.52

key component in our MxGNN approach, we use the same hyperparameter as in our MxGNN approach. Regarding the hyperparameters of GraphSAGE, SortPool, gPool, SAGPool, and GIN, we follow the same experimental setups described in their original papers. Besides, we adopt the widely used evaluation metric, i.e., accuracy, for graph classification to evaluate the performance. The final test fold score is obtained as the mean of three runs with unfavorable random weight initializations. All models are trained using one NVIDIA GeForce RTX 2080 Ti GPU.

MxGNN Configurations The convolution GNN model used is the original Graph Convolution Network model [7]. The pooling GNN model used is the DiffPool [6] model. The model configurations for convolution GNN and pooling GNN are the same as DiffPool. Besides, our MxGNN comprises multiple graph convolution networks with different sets of hyperparameters to learn graph features from different perspectives. We concurrently run 3 graph convolution networks. The a priori graph properties vector used in our model contains 3 graph properties including the number of nodes, the number of edges, and the average node degree. These a priori graph properties for each graph are prepared before the training starts. We train our networks using Adam optimizer with a learning rate of 0.001.

B. ACCURACY RESULTS ON GRAPH CLASSIFICATION

We evaluate our proposed MxGNN on eight benchmark datasets and compare with several state-of-the-art baselines. The baseline method proposed in [35] is structure-agnostic and only exploits node features. The accuracy results are reported in Table 4 where the best results are shown in bold. For gPool, SAGPool, and GIN, we use their PyTorch Geometric implementations to test graph classification accuracy. Regarding the SAGPool baseline [9], we meet OutOfResource error when processing the RDT-M12K dataset, and we denote this case by ‘OOD’. From the table, we observe that our proposed MxGNN approach obtains the best performance on 5 out of 6 benchmark datasets. GIN and SAGPool are

better than MxGNN on the COLLAB dataset. But MxGNN performs better than GIN and SAGPool on the other datasets. The DiffPool model is the most similar one to our MxGNN. We extend the DiffPool model by employing the multiplex structure. Our model outperforms DiffPool by an average of 1.79%, which can be attributed to the effect of the multiplex structure.

C. EFFECTS OF MULTIPLEX CONVOLUTION

Our motivation for this work is to utilize a multiplex hierarchical structure to deal with the diversity and complexity challenges in graph representation learning. Multiple graph convolutional networks with different sets of hyperparameters are used to learn node representations. The node embedding size, as a hyperparameter in GCN, plays an important role in determining the quality of node representation. We vary the node embedding sizes in different GCN networks.

To verify the effectiveness of multiplex convolution, we run our MxPool with a single convolution network and single pooling network (SCSP), multiple convolution networks and single pooling network (MCSP), respectively. The accuracy results on 5 datasets for graph classification are shown in Table 5. Since the suitable node embedding sizes are not consistent for different datasets, we use c_1, c_2, c_3 to denote three different graph convolution parameters (i.e., node embedding size). Note that, they are different for different datasets. We have put the detailed parameter settings on our GitHub project page.

From the table, we observe that the multiplex structure significantly improves performance over the singular structure. By fixing the pooling network with p , multiplexing three convolution networks with hyperparameters $[c_1|c_2|c_3]$ consistently performs better than using single convolution network with either c_1, c_2 , or c_3 . Multiplex convolution plays an important role in improving performance.

To verify the effect of pooling rate on graph classification, we set the pooling rate to different values, as shown in Table 6.

TABLE 5. Effects of multiplex convolution.

Variations		D&D	ENZYMES	PROTEINS	NCI109	COLLAB
SCSP	$c_1 * p$	80.01	61.42	75.48	80.07	71.31
	$c_2 * p$	79.33	62.17	74.47	80.10	71.33
	$c_3 * p$	78.75	61.44	75.14	78.49	71.14
MCSP	$[c_1 c_2 c_3] * p$	80.47	68.22	76.30	81.89	73.43

TABLE 6. Effects of pooling ratio.

Variations		D&D	ENZYMES	PROTEINS	NCI109	COLLAB
MCSP	$[c_1 c_2 c_3] * p_1$	80.47	68.22	76.30	81.89	73.43
	$[c_1 c_2 c_3] * p_2$	80.22	67.25	76.25	81.89	72.34
	$[c_1 c_2 c_3] * p_3$	79.33	68.17	76.25	81.5	73.33

Since the range of pooling ratio suitable for each dataset is not necessarily the same, we use P and its subscript to represent it fuzzily. It can be seen from the experimental results that the pooling ratio has little effect on the experimental results under the condition of multiple convolution networks and single pooling network (MCSP), which indicates that multi convolution network plays a positive role in improving the performance of graph classification.

D. DISTRIBUTION OF ATTENTION WEIGHTS

To learn graph features from a graphs dataset where graphs exhibit very different properties, MxGNN employs multiple convolution networks. By exploiting the a prior graph properties (e.g., number of nodes, number of edges, and average degree), MxGNN assigns different attention weights to different convolution networks. In order to verify if the attention weights can be learned through our end-to-end structure, we first run the MxGNN training process and record the learned attention weights after the training process is converged.

The recorded attention weight distribution of multiple convolution networks for the NCI109 dataset is shown in Figure 3. We group the attention weight results based on the graph's node number/edge number/average degree and show the average attention weight of each group in the figure. That is, we show the attention weight distribution for graphs with different sizes so that we can see if the weight distribution varies when processing different size graphs. The attention weight portion of each convolution network for graphs with the different number of nodes is depicted in Figure 3a, where conv1, conv2, and conv3 indicate the three convolutional networks with small, medium, and large node embedding size, respectively. We observe that the graphs with different node numbers indeed obtain different attention weight distributions. Smaller graphs (e.g., graphs with node number 1-10) prefer the first convolutional network with small node embedding size, while larger graphs (e.g., graphs with node number 90-100) prefer the third network with large node embedding size. With the increasing size of graphs, the attention weight to the third convolutional network becomes bigger and bigger. Similarly, Figure 3b clearly shows that the attention weight to each convolution network is relevant to the number of edges of the input graph. The

TABLE 7. Effect of number of convolution.

No. of networks	1	2	3	4	5
ENZYMES	62.53	59.83	68.22	65.33	61.71
PROTEINS	76.25	76.31	78.40	77.36	76.64
COLLAB	71.78	75.95	77.20	76.77	76.07

average node degree of a graph, as another common graph property, also effects the preference of different convolutional networks. As shown in Figure 3c, the attention weight to each convolution network is relevant to the average node degree of the input graph. Besides the above three graph properties, there are also other graph properties that may take effect, e.g., diameter, clustering coefficient, number of triangles, etc.

To clearly see how the attention weight is distributed among different GCNs, we further perform such a weight distribution analysis on the ENZYMES dataset. The attention weight distribution of multiple convolution networks is shown in Figure 4. From these figures, we can see similar weight distributions from those observed on the NCI109 dataset. This proved that our MxGNN integrates multiple networks with different sets of hyperparameters, and can automatically learn the best weight distribution among different networks to offer the best performance given the improper hyperparameters.

E. NUMBER OF CONVOLUTION

The number of convolution networks is a hyperparameter in MxGNN. In the previous experiments, we use a fixed number of convolution networks to show the performance. In this experiment, we vary the number of convolution networks from 1 to 5 and evaluate the performance. The graph classification accuracy results on ENZYMES, PROTEINS, and COLLAB datasets are listed in Table 7. When using a single convolution network, MxGNN downgrades to the exact Diff-Pool. From the table, we can see that the best performance is achieved when the number is set around 3-4. As the number is increased larger, the performance is reduced. This may be because that too many networks with a large number of parameters result in overfitting problems.

F. COMPUTATION COMPLEXITY ANALYSIS

We compare the accuracy convergence curves of train sets with time when GraphSage [3] with max-pooling,

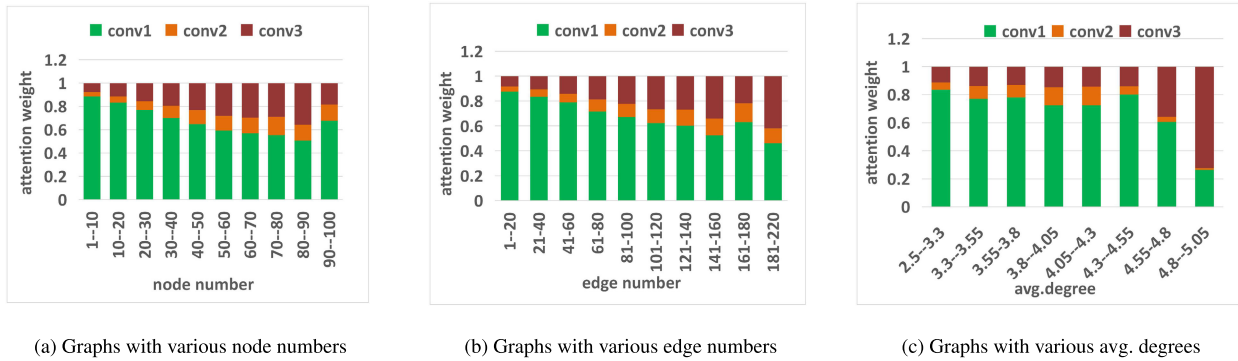


FIGURE 3. Attention weight distribution of multiple convolutional networks (NCI109).

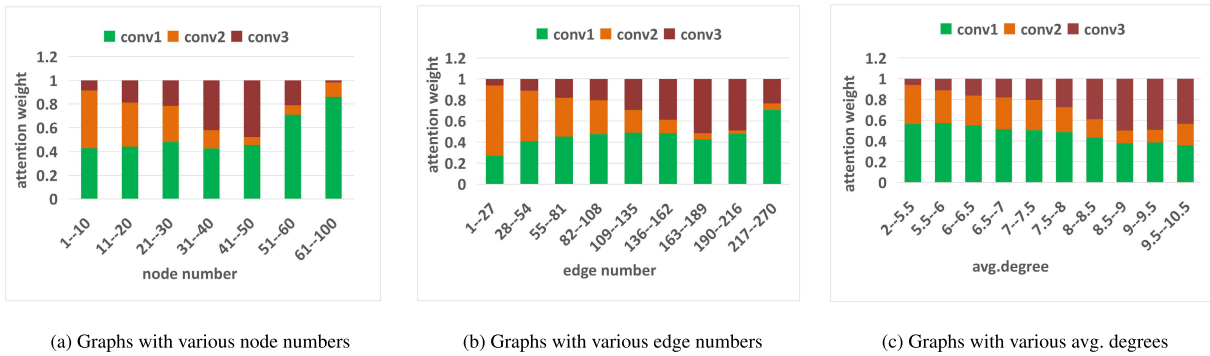


FIGURE 4. Attention weight distribution of multiple convolutional networks (ENZYMES).

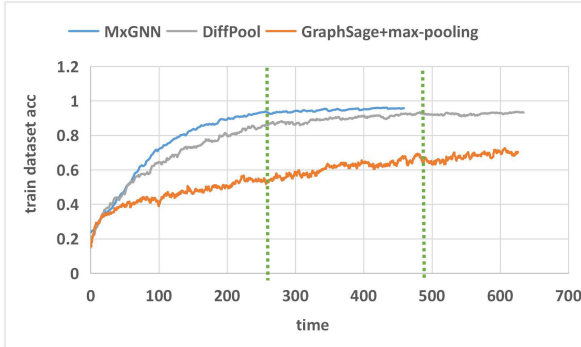


FIGURE 5. Convergence curves of training sets of ENZYMES dataset under MxGNN, Diffpool and, GraphSage.

DiffPool [6] and MxGNN perform graph classification task on the ENZYMES dataset. As shown in Figure 5, MxGNN converges in about 280 seconds, DiffPool converges in about 600 seconds and Graphsage with max-pooling does not converge at 600 seconds. The convergence speed of MxGNN is two times more than that of DiffPool. Therefore, although MxGNN has a multi convolution structure, it will bring extra computational overhead on the surface. However, MxGNN converges faster. We recorded the time required for the model to run once. The running time of MxGNN is 65 minutes and that of DiffPool is 50 minutes, which shows that MxGNN will not increase the computational complexity greatly. And the accuracy of graph classification is 64.17% and 60.24% respectively, MxGNN significantly improves the accuracy of graph classification.

V. CONCLUSION

In this paper, we proposed a simple but effective multiplex GNN architecture MxGNN for hierarchical graph representation learning. MxGNN comprises multiple graph convolution networks to learn node-level representations and use graph pooling networks to coarsen the graph. The a priori graph properties are employed to assign the attention weight to the convolution network so that the diversity challenge of graph representation learning can be well addressed. Our results show that MxGNN has gained performance improvement over the state-of-the-art graph representation learning methods. In future work, we intend to design a variant of MxGNN, which adds a multiplex pooling layer after the multiplex convolution layer. And design unpooling layers to form an encoder-decoder learning structure to deal with node classification tasks and link prediction tasks.

REFERENCES

- [1] Y. LeCun and Y. Bengio, "Convolutional networks for images, speech, and time series," in *The Handbook of Brain Theory and Neural Networks*, M. A. Arbib, Ed. Cambridge, MA, USA: MIT Press, 1998, pp. 255–258.
- [2] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains," *IEEE Signal Process. Mag.*, vol. 30, no. 3, pp. 83–98, May 2013.
- [3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 1025–1035.
- [4] Y. Bengio and Y. LeCun, "Spectral networks and locally connected networks on graphs," in *Proc. 2nd Int. Conf. Learn. Represent. (ICLR)*, 2014, pp. 1–14.

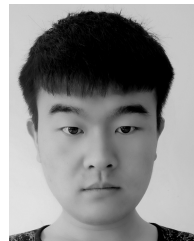
- [5] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, and H.-P. Kriegel, "Protein function prediction via graph kernels," *Bioinformatics*, vol. 21, pp. 47–56, Jun. 2005.
- [6] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2018, pp. 4805–4815.
- [7] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1–13.
- [8] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2018, pp. 4438–4445.
- [9] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 3734–3743.
- [10] H. Gao and S. Ji, "Graph U-nets," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 2083–2092.
- [11] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. 7th Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–17.
- [12] D. Duvenaud, R. Gómez-Bombarelli, D. Maclaurin, T. Hirzel, R. P. Adams, A. Aspuru-Guzik, and J. Aguilera-Iparraguirre, "Convolutional networks on graphs for learning molecular fingerprints," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2015, pp. 2224–2232.
- [13] N. Shervashidze, S. Vishwanathan, T. Petri, K. Mehlhorn, and K. Borgwardt, "Efficient graphlet kernels for large graph comparison," in *Proc. Artif. Intell. Statist.*, 2009, pp. 488–495.
- [14] J. B. Lee, R. Rossi, and X. Kong, "Graph classification using structural attention," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 1666–1674.
- [15] T. Ma, H. Wang, L. Zhang, Y. Tian, and N. Al-Nabhan, "Graph classification based on structural features of significant nodes and spatial convolutional neural networks," *Neurocomputing*, vol. 423, pp. 639–650, Jan. 2021, doi: [10.1016/j.neucom.2020.10.060](https://doi.org/10.1016/j.neucom.2020.10.060).
- [16] F. Diehl, "Edge contraction pooling for graph neural networks," May 2019.
- [17] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. Extended Semantic Web Conf.*, 2018, pp. 593–607.
- [18] H. Dai, B. Dai, and L. Song, "Discriminative embeddings of latent variable models for structured data," in *Proc. 33rd Int. Conf. Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 2702–2711.
- [19] J. Wang, C. Jiang, H. Zhang, Y. Ren, K.-C. Chen, and L. Hanzo, "Thirty years of machine learning: The road to Pareto-optimal wireless networks," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 1472–1514, 3rd Quart., 2020.
- [20] Z. Guo and H. Wang, "A deep graph neural network-based mechanism for social recommendations," *IEEE Trans. Ind. Informat.*, vol. 17, no. 4, pp. 2776–2783, Apr. 2021.
- [21] Z. Guo, K. Yu, Y. Li, G. Srivastava, and J. C.-W. Lin, "Deep learning-embedded social Internet of Things for ambiguity-aware social recommendations," *IEEE Trans. Netw. Sci. Eng.*, early access, Jan. 5, 2021, doi: [10.1109/TNSE.2021.3049262](https://doi.org/10.1109/TNSE.2021.3049262).
- [22] Z. Guo, K. Yu, A. Jolfaei, A. K. Bashir, A. O. Almagrabi, and N. Kumar, "A fuzzy detection system for rumors through explainable adaptive learning," *IEEE Trans. Fuzzy Syst.*, early access, Jan. 15, 2021, doi: [10.1109/TFUZZ.2021.3052109](https://doi.org/10.1109/TFUZZ.2021.3052109).
- [23] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2016, pp. 3844–3852.
- [24] R. Li, S. Wang, F. Zhu, and J. Huang, "Adaptive graph convolutional neural networks," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2018, pp. 3546–3553.
- [25] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015, p. 41.
- [26] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1263–1272.
- [27] S. Wu, Y. Zhang, C. Gao, K. Bian, and B. Cui, "GARG: Anonymous recommendation of point-of-interest in mobile networks by graph convolution network," *Data Sci. Eng.*, vol. 5, no. 4, pp. 433–447, Dec. 2020.
- [28] K.-P. Yu, L. Tan, M. Aloqaily, H. Yang, and Y. Jararweh, "Blockchain-enhanced data sharing with traceable and direct revocation in IIoT," *IEEE Trans. Ind. Informat.*, early access, Jan. 5, 2021, doi: [10.1109/TII.2021.3049141](https://doi.org/10.1109/TII.2021.3049141).
- [29] Y. Ma, S. Wang, C. C. Aggarwal, and J. Tang, "Graph convolutional networks with EigenPooling," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 723–731.
- [30] R. L. Murphy, B. Srinivasan, V. Rao, and B. Ribeiro, "Relational pooling for graph representations," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 4663–4673.
- [31] H. Yuan and S. Ji, "Structpool: Structured graph pooling via conditional random fields," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–12.
- [32] P. D. Dobson and A. J. Doig, "Distinguishing enzyme structures from non-enzymes without alignments," *J. Mol. Biol.*, vol. 330, no. 4, pp. 771–783, Jul. 2003.
- [33] N. Wale, I. A. Watson, and G. Karypis, "Comparison of descriptor spaces for chemical compound retrieval and classification," *Knowl. Inf. Syst.*, vol. 14, no. 3, pp. 347–375, Mar. 2008.
- [34] P. Yanardag and S. Vishwanathan, "A structural smoothing framework for robust graph comparison," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2134–2142.
- [35] F. Errica, M. Podda, D. Bacciu, and A. Micheli, "A fair comparison of graph neural networks for graph classification," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–14.



YANYAN LIANG received the bachelor's degree from Qingdao University, China, in 2014. She is currently pursuing the master's degree with Northeast University, China. She is also following her tutor Prof. Y. Zhang to study graph neural networks.



YANFENG ZHANG (Member, IEEE) received the Ph.D. degree in computer science from Northeastern University, China, in 2012. He is currently an Associate Professor with Northeastern University. His research interests include distributed systems and big data processing. He has published many articles in the above areas. His paper in Soc 2011 was honored with Paper of Distinction.



DECHAO GAO received the B.Eng. degree in computer science and technology from the Northeast University (NEU), Shenyang, China, in 2020. He is currently pursuing the degree. He is also following his tutor Prof. Y. Zhang to study graph neural networks.



QIAN XU is currently pursuing the degree with the School of Computer Science and Engineering, Northeastern University. Her research interests include distributed systems and big data processing.

...